

CAN Simulator

Od prostego GUI do zaawansowanej platformy
diagnostycznej

Projekt: MagistralaCAN

Opracowanie: Asystent AI na podstawie dokumentacji projektowej

25 kwietnia 2026

Spis treści

1	Wprowadzenie	5
1.1	Geneza projektu	5
1.2	Zakres modernizacji	5
1.3	Struktura dokumentu	6
2	Reorganizacja interfejsu użytkownika	7
2.1	Problem: przeładowany pasek zakładek	7
2.2	Rozwiązanie: Sidebar z kategoriami	7
2.3	Implementacja	8
2.4	Efekt końcowy	8
3	Integracja plików DBC	9
3.1	Wprowadzenie do DBC	9
3.2	Moduł <code>dbc_manager.py</code>	9
3.3	Integracja z GUI	10
3.3.1	Zakładka DBC Manager	10
3.3.2	Rozszerzenie Sniffera CAN	10
3.3.3	Walidacja zakresów	10
3.4	Wykorzystanie DBC w innych zakładkach	10
4	Zdalny monitoring przez WebSocket	11
4.1	Wymagania	11
4.2	Architektura serwera	11
4.3	Integracja z GUI	11
4.4	Rozsyłanie ramek	12
5	Dwukierunkowy mostek vCAN	13
5.1	Koncepcja	13
5.2	Implementacja po stronie klienta (Wojtek)	13
5.3	Implementacja po stronie serwera (Ty)	13
5.4	Konfiguracja GUI	14
6	Wykrywanie anomalii w czasie rzeczywistym	15
6.1	Motywacja	15
6.2	Autoenkoder online	15
6.3	Integracja z GUI	16
6.4	Uwagi praktyczne	16

7	Interfejs webowy (dashboard)	17
7.1	Cel	17
7.2	Architektura	17
7.3	Funkcjonalność dashboardu	17
7.4	Implementacja serwera HTTP	18
7.5	Konfiguracja w GUI	18
8	Powiadomienia Telegram i bezpieczeństwo	19
8.1	Powiadomienia Telegram	19
8.2	Autoryzacja na poziomie ID	19
8.3	Logowanie zdalnych operacji	20
9	Nagrywanie i odtwarzanie sesji CAN	21
9.1	Funkcjonalność	21
9.2	Format pliku	21
9.3	Implementacja	21
10	Testowanie i ciągła integracja	23
10.1	Wprowadzenie	23
10.2	Struktura testów	23
10.3	Uruchamianie testów	23
10.4	GitHub Actions	24
11	Wdrażanie z Dockerem	25
11.1	Dlaczego Docker?	25
11.2	Pliki konfiguracyjne	25
11.3	Budowa obrazu	25
11.4	Uruchomienie	26
12	Mapa drogowa i wizja przyszłości	27
12.1	Podsumowanie dotychczasowych osiągnięć	27
12.2	Plany krótkoterminowe (Faza 2)	27
12.3	Plany średnioterminowe (Faza 3)	27
12.4	Plany długoterminowe (Faza 4)	28
12.5	Wizja przyszłości (Faza 5)	28
12.6	Zakończenie	28
A	Kod źródłowy mostka vCAN (bridge_client.py)	29
B	Kod źródłowy serwera WebSocket (server.py)	35

Rozdział 1

Wprowadzenie

1.1 Geneza projektu

Projekt **CAN Simulator** narodził się jako proste narzędzie desktopowe napisane w Pythonie z wykorzystaniem biblioteki Tkinter. Jego pierwotnym celem była symulacja ruchu na magistrali CAN (Controller Area Network) poprzez odtwarzanie wcześniej zapisanych logów oraz umożliwienie ręcznego wysyłania pojedynczych ramek. Z czasem, wraz z rosnącymi potrzebami użytkownika – inżyniera diagnostyki (nazwijmy go Wojtkiem) – aplikacja ewoluowała w kierunku zaawansowanej platformy do zdalnego monitorowania, analizy danych i uczenia maszynowego.

1.2 Zakres modernizacji

Niniejszy dokument opisuje kompleksową modernizację oprogramowania, która została przeprowadzona w kilkunastu etapach. Każdy z nich wprowadzał nowe funkcjonalności, poprawiał istniejące niedociągnięcia lub zwiększał stabilność i bezpieczeństwo systemu. Poniżej przedstawiono kluczowe obszary objęte zmianami:

- Reorganizacja interfejsu użytkownika (GUI) – zastąpienie poziomego paska zakładek boczną listą kategorii.
- Pełna integracja plików DBC (CAN Database) – dekodowanie sygnałów, walidacja zakresów, eksport.
- Implementacja zdalnego monitoringu przez WebSocket (w tym szyfrowane WSS) wraz z dwukierunkowym mostkiem vCAN.
- Dodanie zaawansowanych funkcji uczenia maszynowego: prognozowanie LSTM, wykrywanie anomalii w czasie rzeczywistym (autoenkoder).
- Stworzenie lekkiego interfejsu webowego (dashboard HTML/JS) oraz powiadomień Telegram.
- Wprowadzenie testów jednostkowych i ciągłej integracji (GitHub Actions).
- Przygotowanie środowiska Docker do łatwego wdrażania.

1.3 Struktura dokumentu

Dokument podzielony jest na rozdziały, z których każdy odpowiada jednemu z głównych etapów modernizacji. W poszczególnych rozdziałach omówiono:

1. Motywację stojącą za daną zmianą.
2. Architekturę i kluczowe komponenty.
3. Fragmenty kodu ilustrujące implementację.
4. Sposób integracji z istniejącą bazą kodu.

Dodatki zawierają pełne listingi najważniejszych plików źródłowych, takich jak mostek vCAN (`bridge_client.py`) oraz serwer WebSocket (`server.py`).

Rozdział 2

Reorganizacja interfejsu uÅijtkownika

2.1 Problem: przeÅćadowany pasek zakÅćadek

Pierwotny interfejs CAN Simulatora opieraÅć siÄŹ na komponencie `ttk.Notebook`, ktÅşy zawieraÅć kilkanaÅŹcie zakÅćadek. W miarÄŹ dodawania nowych funkcji (serwer TCP, zaawansowane ML, DBC Manager, zdalny monitoring, mostek vCAN) pasek zakÅćadek stawaÅć siÄŹ nieczytelny, a nawigacja âŹ uciÄŹÅijliwa.

2.2 RozwiÄŹzanie: Sidebar z kategoriami

Zdecydowano siÄŹ na reorganizacjÄŹ ukÅćadu okna gÅćÅşwnego poprzez zastÄŹpienie Notebooka strukturÄŹ zÅćoÅijonÄŹ z:

- Lewego panelu (sidebar) âŹ tk.Listbox wyÅŹwietlajÄŹcego listÄŹ kategorii.
- Prawego panelu âŹ kontenera `ttk.Frame`, w ktÅşym dynamicznie wyÅŹwietlana jest zawartoÅŹÄŹ wybranej kategorii.

Kategorie pogrupowano tematycznie:

1. PoÅćÄŹczenie CAN
2. Podstawowe narzÄŹdzia (Odtwarzanie, Symulacja, Ramki bÅćÄŹdu, Tryb krokowy, WysyÅćanie rÄŹczne)
3. Wyszukiwanie i analiza (Wyszukiwanie binarne, Kreator, Sniffer)
4. Wizualizacja i ML (Wykresy, Zaawansowane ML, Analiza wzorcÅşw, Analiza ML, Anomalie CAN)
5. SieŹ i zdalny dostÄŹp (Serwer TCP, Zdalny monitoring, Generator ruchu, Mostek vCAN, Nagrywanie sesji)
6. Makra
7. DBC Manager

2.3 Implementacja

W pliku `gui/app.py` wprowadzono następujące zmiany:

```
1 main_panel = ttk.PanedWindow(self.root, orient=tk.HORIZONTAL)
2 main_panel.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)
3
4 self.sidebar = tk.Listbox(main_panel, width=25, exportselection=
5     False)
6 self.sidebar.pack(side=tk.LEFT, fill=tk.Y)
7 main_panel.add(self.sidebar, weight=0)
8
9 self.content_frame = ttk.Frame(main_panel)
10 self.content_frame.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True)
11 main_panel.add(self.content_frame, weight=1)
12
13 categories = [
14     ("Połączenie CAN", self._create_connection_frame),
15     ("Podstawowe narzędzia", self._create_basic_tools_frame),
16     # ... pozostałe kategorie
17 ]
18
19 for idx, (cat_name, setup_func) in enumerate(categories):
20     self.sidebar.insert(tk.END, cat_name)
21     frame = ttk.Frame(self.content_frame)
22     self.category_frames[cat_name] = frame
23     setup_func(frame)
24
25 self.sidebar.bind('<<ListboxSelect>>', self._on_category_select)
26 self.sidebar.selection_set(0)
27 self._on_category_select()
```

Listing 2.1: Fragment `_create_widgets` z nowym sidebarem

2.4 Efekt końcowy

Nowy interfejs jest znacznie bardziej przejrzysty. Użytkownik jednym kliknięciem przełącza się między głównymi obszarami funkcjonalnymi, a liczba jednocześnie widocznych kontrolki została zredukowana, co poprawia komfort pracy.

Rozdział 3

Integracja plików DBC

3.1 Wprowadzenie do DBC

Pliki DBC (CAN Database) są standardowym formatem opisu komunikatów i sygnałów w sieciach CAN. Zawierają definicje identyfikatorów ramek, nazwy sygnałów, ich położenie bitowe, skalowanie, jednostki oraz wartości graniczne. Integracja DBC z CAN Simulatorem umożliwia pracę z nazwami sygnałów zamiast surowych bajtów, co radykalnie ułatwia analizę danych.

3.2 Moduł `dbc_manager.py`

Centralnym elementem obsługi DBC jest klasa `DBCManager`, wykorzystująca bibliotekę `cantools`. Odpowiada ona za:

- Wczytywanie pliku `.dbc`.
- Dekodowanie ramek na podstawie identyfikatora i danych.
- Udostępnianie listy dostępnych sygnałów w formacie `"Message.Signal"`.
- Eksport listy sygnałów do pliku CSV.

```
1 def decode_frame(self, frame_id: int, data: bytes):
2     if not self.db:
3         return None
4     try:
5         message = self.db.get_message_by_frame_id(frame_id)
6         decoded = message.decode(data)
7         limits = {}
8         for sig in message.signals:
9             limits[sig.name] = (sig.minimum, sig.maximum)
10        return {
11            "message_name": message.name,
12            "signals": decoded,
13            "limits": limits
14        }
15    except Exception:
16        return None
```

Listing 3.1: Metoda `decode_frame` w `dbc_manager.py`

3.3 Integracja z GUI

3.3.1 Zakładka DBC Manager

Nowa zakładka (w kategorii głównej) umożliwia wczytanie pliku DBC, wyświetlenie listy sygnałów oraz podgląd ich szczegółów (min, max, jednostka, komentarz). Dodano również przycisk eksportu do CSV.

3.3.2 Rozszerzenie Sniffera CAN

W tabeli sniffera dodano kolumnę "Zdekodowane (DBC)". W postaci przetworzonej ramki wywoływana jest metoda `decode_frame`, a wynik formatowany jest do czytelnej postaci (np. "EngineSpeed=2500.00, CoolantTemp=90.00").

3.3.3 Walidacja zakresów

Jeżeli wartość sygnału wykracza poza zdefiniowane minimum lub maksimum, wiersz w tabeli podświetlany jest na czerwono, a w logu pojawia się ostrzeżenie. Funkcja ta wykorzystuje słownik `limits` zwracany przez `decode_frame`.

3.4 Wykorzystanie DBC w innych zakładkach

- **Wykresy** – dodano rozwijaną listę sygnałów DBC; wybór sygnału automatycznie ustawia identyfikator i indeks bajtu.
- **Zaawansowane ML (Prognozowanie)** – analogiczny mechanizm wyboru sygnału z DBC.

Rozdział 4

Zdalny monitoring przez WebSocket

4.1 Wymagania

Aby umożliwić Wojtkowi zdalny podgląd ruchu CAN, zaimplementowano serwer WebSocket, który rozsyła ramki do wszystkich połączonych klientów. Serwer obsługuje:

- Opcjonalny token autoryzacyjny (pierwsza wiadomość od klienta).
- Szyfrowanie TLS (WSS) – możliwość wskazania certyfikatu i klucza prywatnego.
- Generowanie samopodpisanego certyfikatu testowego.

4.2 Architektura serwera

Serwer został zaimplementowany jako klasa `CANWebSocketServer` w module `broadcaster/server`. Wykorzystuje bibliotekę `websockets` i działa w osobnej wątku asynchronicznej (`asyncio`), uruchamianej w dedykowanym wątku, aby nie blokować głównego wątku GUI.

```
1 async def _handler(self, websocket: WebSocketServerProtocol):
2     if self.token:
3         msg = await asyncio.wait_for(websocket.recv(), timeout=5.0)
4         if msg != self.token:
5             await websocket.close(1008, "Invalid token")
6             return
7     self._clients.add(websocket)
8     try:
9         await websocket.wait_closed()
10    finally:
11        self._clients.remove(websocket)
```

Listing 4.1: Uproszczony handler połączenia

4.3 Integracja z GUI

W zakładce "Zdalny monitoring" dodano panel umożliwiający:

- Ustawienie portu i tokena.
- Wczytanie WSS i wskazanie pliku certyfikatu.
- Generowanie certyfikatu testowego (OpenSSL).
- Uruchomienie/zatrzymanie serwera.
- Podgląd logów i liczby aktywnych klientów.

4.4 Rozsyłanie ramek

Aby serwer mógł transmitować rzeczywisty ruch CAN, został podpięty do głównego interfejsu CAN aplikacji. W tym celu rozszerzono klasę `CanInterface` o system callbacków, który każdą odebraną ramkę wywołuje listę zarejestrowanych funkcji, a serwer dodaje w czasie callback, który wywołuje `broadcast_frame_async`.

```
1 def add_frame_callback(self, callback):
2     if callback not in self.frame_callbacks:
3         self.frame_callbacks.append(callback)
4
5 # w pętli odbioru:
6 for cb in self.frame_callbacks:
7     cb({'id': hex(can_id), 'data': list(data), 'is_extended':
        is_extended, 'timestamp': time.time()})
```

Listing 4.2: Callback w `CanInterface`

Rozdział 5

Dwukierunkowy mostek vCAN

5.1 Koncepcja

Początkowo mostek vCAN (`bridge_client.py`) jedynie odbierał ramki z WebSocket i wstrzykiwał je do wirtualnego interfejsu `vcan0`. Aby Wojtek mógł wysyłać ramki (np. zapytania diagnostyczne) na fizyczny magistralę CAN, rozszerzono mostek o komunikację dwukierunkową.

5.2 Implementacja po stronie klienta (Wojtek)

W klasie `VCanBridge` dodano wątek `_tx_loop`, który nasłuchuje na magistrali `vcan0` (przy użyciu `ThreadSafeBus` z biblioteki `python-can`). Każda odebrana ramka, która nie pochodzi z WebSocket (flaga `sending_to_vcan`), jest konwertowana do formatu JSON i wysyłana z powrotem do serwera.

```
1 def _tx_loop(self):
2     while self.running:
3         msg = self.bus.recv(timeout=0.5)
4         if msg is None or self.sending_to_vcan:
5             continue
6         if self.filter_ids and msg.arbitration_id not in self.filter_ids:
7             continue
8         if self.ws and self.ws.sock and self.ws.sock.connected:
9             frame = {
10                 "id": hex(msg.arbitration_id),
11                 "data": list(msg.data),
12                 "is_extended": msg.is_extended_id
13             }
14             self.ws.send(json.dumps(frame))
```

Listing 5.1: Wątek nadawczy w `bridge_client.py`

5.3 Implementacja po stronie serwera (Ty)

Serwer WebSocket został rozszerzony o metodę `_handle_incoming_frame`, która wywoływana jest dla każdej wiadomości otrzymanej od klienta. Po sprawdze-

niu filtra ID (opcjonalnego), ramka jest wysyłana na fizyczny interfejs CAN za pośrednictwem referencji do `CanInterface`.

```
1 async def _handle_incoming_frame(self, websocket, message):
2     data = json.loads(message)
3     can_id = int(data['id'], 16)
4     if can_id not in self.incoming_filter_ids:
5         return
6     payload = bytes(data['data'])
7     if self.can_interface and self.can_interface.connected:
8         self.can_interface.send_frame(can_id, payload, data.get('
            is_extended', False))
```

Listing 5.2: Obsługa ramki przychodzącej w `server.py`

5.4 Konfiguracja GUI

W zakładce "Zdalny monitoring" dodano pole "Dozwolone ID dla klientów" (filtrowanie ramek przychodzących) oraz opcję logowania zdalnych operacji do pliku.

Rozdział 6

Wykrywanie anomalii w czasie rzeczywistym

6.1 Motywacja

Tradycyjne monitorowanie magistrali CAN opiera się na sprawdzaniu wartości sygnałów względem statycznych zakresów (DBC). Jednak wiele usterek objawia się nietypowymi sekwencjami ramek lub korelacjami między sygnałami, które trudno wychwycić prostymi regułami. Rozwiązaniem jest zastosowanie uczenia maszynowego – konkretnie autoenkodera – do wykrywania anomalii w czasie rzeczywistym.

6.2 Autoenkoder online

Zaimplementowano klasę `CANAnomalyDetector` w module `ml/advanced_models.py`. Działająca ona w następujący sposób:

1. Zbiera okno ostatnich N ramek (domyślnie 10).
2. Wyodrębnia cechy: znormalizowany identyfikator CAN oraz wartości bajtów danych.
3. W fazie treningu (pierwsze 100 okien) uczy się rekonstruować normalny ruch.
4. Po zakończeniu treningu dla każdego nowego okna oblicza błąd rekonstrukcji (MSE).
5. Jeśli błąd przekracza próg (np. 95. percentyl błędów treningowych), ramka oznaczana jest jako anomalia.

```
1 def process_frame(self, frame):
2     feats = self._extract_features(frame)
3     self.buffer.append(feats)
4     if not self.is_trained:
5         if len(self.buffer) >= self.window_size:
6             # Trenuj model...
```

```

7         self._train_model()
8         return None
9     X = np.array([np.concatenate(self.buffer[-self.window_size:]))
10    X_scaled = self.scaler.transform(X)
11    X_pred = self.model.predict(X_scaled, verbose=0)
12    mse = np.mean(np.square(X_scaled - X_pred))
13    return mse

```

Listing 6.1: Fragment klasy CANAnomalyDetector

6.3 Integracja z GUI

Dodano nową zakładkę "Anomalie CAN" w kategorii "Wizualizacja i ML". Zakładka zawiera:

- Panel sterowania (wielkość okna, próg percentyla).
- Wykres anomaly score aktualizowany na żywo.
- Okno logu wyświetlające wykryte anomalie.

Po uruchomieniu monitorowania, detektor rejestruje callback w `CanInterface` i zaczyna analizować każdą ramkę. Przekroczenie progu powoduje wpis do logu oraz (opcjonalnie) wysłanie powiadomienia Telegram.

6.4 Uwagi praktyczne

Model autoenkodera zaimplementowano przy użyciu TensorFlow/Keras. Ze względu na konieczność szybkiego treningu online, sieć ma niewielką architekturę (32-16-8-16-32). W testach laboratoryjnych osiąga zadowalającą skuteczność w wykrywaniu symulowanych usterek (np. zanik sygnału, skokowa zmiana wartości).

Rozdział 7

Interfejs webowy (dashboard)

7.1 Cel

Aby umożliwić monitorowanie magistrali CAN z dowolnego urządzenia w sieci lokalnej (lub przez internet) bez konieczności instalowania Pythona, stworzono lekki dashboard HTML/JavaScript. Komunikuje się on z istniejącym serwerem WebSocket i wyświetla dane w czasie rzeczywistym.

7.2 Architektura

Dashboard składa się z trzech plików statycznych:

- `index.html` – struktura strony.
- `style.css` – style wizualne.
- `dashboard.js` – logika klienta WebSocket i rysowanie wykresów.

Pliki te są serwowane przez dodatkowy serwer HTTP uruchamiany w ramach `CANWebSocketServer` (przy użyciu biblioteki `aiohttp`). Serwer HTTP nasłuchuje na konfigurowalnym porcie (domyślnie 8080).

7.3 Funkcjonalność dashboardu

- Połączenie z serwerem WebSocket (adres można zmienić).
- Tabela ostatnich ramek CAN (ID, dane hex, czas).
- Wyświetlanie zdekodowanych wartości DBC (jeśli serwer ma wczytany plik DBC – funkcjonalność można rozszerzyć).
- Prosty wykres wartości wybranego bajtu sygnału (rysowany na elemencie `<canvas>`).

7.4 Implementacja serwera HTTP

W klasie `CANWebSocketServer` dodano metodę `_create_http_app`, która tworzy aplikację `aihttp` serwującą pliki z katalogu `web_dashboard`. W metodzie `start()` uruchamiany jest runner HTTP, jeżeli opcja `enable_http` jest włączona.

```
1 if self.enable_http:
2     self.http_app = self._create_http_app()
3     runner = web.AppRunner(self.http_app)
4     await runner.setup()
5     site = web.TCPSite(runner, self.host, self.http_port)
6     await site.start()
7     self.http_runner = runner
```

Listing 7.1: Fragment `start()` z obsługą HTTP

7.5 Konfiguracja w GUI

W zakładce "Zdalny monitoring" dodano checkbox "Uruchom interfejs webowy" oraz pole do wpisania portu HTTP. Użytkownik może w ten sposób łatwo udostępnić dashboard w sieci.

Rozdział 8

Powiadomienia Telegram i bezpieczeństwo

8.1 Powiadomienia Telegram

Aby Wojtek nie musiał stale obserwować ekranu, zaimplementowano integrację z komunikatorem Telegram. Serwer CAN może wysyłać powiadomienia o następujących zdarzeniach:

- Nowy klient połączony / rozłączony.
- Brak wysłania ramki na fizyczny CAN.
- Wykrycie wartości sygnału poza zakresem DBC (przy współpracy ze Snifferem).
- Wykrycie anomalii przez autoenkoder (zakładka Anomalie CAN).

Do wysłania wiadomości wykorzystywane jest proste żądanie HTTP GET do API Telegrama (<https://api.telegram.org/bot<TOKEN>/sendMessage>). W GUI dodano pola do wpisania tokena bota oraz identyfikatora czatu (Chat ID).

8.2 Autoryzacja na poziomie ID

Aby zwiększyć bezpieczeństwo dwukierunkowej komunikacji, wprowadzono mechanizm autoryzacji identyfikatorów ramek:

1. Serwer posiada listę `allowed_client_ids` (konfigurowalną w GUI).
2. Podczas nawiązywania połączenia, klient (mostek vCAN) wysyła JSON z listą identyfikatorów, które zamierza wysłać (`{"allowed_ids": [0x123, 0x456]}`).
3. Serwer weryfikuje, czy wszystkie identyfikatory klienta należą do zbioru dozwolonego. Jeśli nie, połączenie jest odrzucane.

8.3 Logowanie zdalnych operacji

Dodano opcję zapisu wszystkich zdalnych operacji (ramki odebrane od klientów, ramki wysłane do klientów) do pliku `remote_operations.log`. Każdy wpis zawiera timestamp oraz adres IP klienta.

```
1 def _log_to_file(self, msg):  
2     with open(self.log_file_path, 'a', encoding='utf-8') as f:  
3         f.write(f"{datetime.now().isoformat()} {msg}\n")
```

Listing 8.1: Metoda `_log_to_file`

Rozdział 9

Nagrywanie i odtwarzanie sesji CAN

9.1 Funkcjonalność

Aby umożliwić Wojtkowi zapisanie całego zdalnego ruchu CAN do późniejszej analizy offline, dodano zakładkę "Nagrywanie sesji" w kategorii "Sieć i zdalny dostęp".

Zakładka umożliwia:

- Wybór pliku wyjściowego (domyślnie nazwa z datą i godziną).
- Ustawienie maksymalnego czasu nagrywania (0 = bez limitu).
- Rozpoczęcie/zatrzymanie nagrywania.
- Szybkie odtworzenie nagrania – plik automatycznie ładowany jest w zakładce "Odtwarzanie pliku".

9.2 Format pliku

Nagrane dane zapisywane są w formacie `candump`, czyli:

```
1 (timestamp) interface id#data
```

Przykład:

```
(1620000000.123456) vcan0 123#DEADBEEF
```

9.3 Implementacja

Nagrywanie wykorzystuje system callbacków `CanInterface`. Po uruchomieniu nagrywania, widget główny rejestruje funkcję `_on_frame`, która dla każdej ramki zapisuje odpowiednią linię do pliku.

```
1 def _on_frame(self, frame):  
2     ts = frame.get('timestamp', time.time())  
3     can_id = int(frame['id'], 16) if isinstance(frame['id'], str)  
4         else frame['id']  
     data_str = ''.join(f'{b:02X}' for b in frame['data'])
```

```
5     line = f"({ts:.6f}) vcan0 {can_id:03X}#{data_str}\n"  
6     self.fd.write(line)
```

Listing 9.1: Callback nagrywania

Licznik zapisanych ramek jest aktualizowany na bieżąco w GUI. Opcjonalny timer (`threading.Timer`) automatycznie zatrzymuje nagrywanie po upływie zadanego czasu.

Rozdział 10

Testowanie i ciągła integracja

10.1 Wprowadzenie

Wraz ze wzrostem złożoności projektu, ręczne testowanie wszystkich funkcji stało się niepraktyczne. Aby zapewnić stabilność i ułatwić dalszy rozwój, wprowadzono automatyczne testy jednostkowe oraz integrację z GitHub Actions.

10.2 Struktura testów

Testy znajdują się w katalogu `tests/` i obejmują:

- `test_dbc_manager.py` – wczytywanie DBC, dekodowanie, zakresy.
- `test_bridge_client.py` – filtrowanie ID, parsowanie listy ID.
- `test_session_manager.py` – zapis i odczyt stanu sesji (mock Tkinter).
- `test_binary_search.py`, `test_cyclic_detector.py` – logika wyszukiwania i detekcji cykli.
- `test_integration.py` – testy integracyjne z wirtualnym CAN (oznaczone jako `@pytest.mark.skip` ze względu na zależność systemową).

10.3 Uruchamianie testów

Do uruchamiania testów służy skrypt `run_tests.sh`, który aktywuje środowisko wirtualne i wywołuje `pytest`. Testy nie wymagają uprawnień roota.

```
1 source venv/bin/activate
2 python -m pytest tests/ -v
```

Listing 10.1: Fragment `run_tests.sh`

10.4 GitHub Actions

W katalogu `.github/workflows/tests.yml` zdefiniowano workflow, który przy każdym pushu do gałęzi `main`:

1. Uruchamia testy na Python 3.10, 3.11 i 3.12.
2. Instaluje zależności systemowe (`can-utils`, `iproute2`) i Pythona.
3. Wykonuje `pytest`.

Dzięki temu każda zmiana jest automatycznie weryfikowana, a ewentualne błędy wykrywane są natychmiast.

Rozdział 11

Wdrażanie z Dockerem

11.1 Dlaczego Docker?

Aby uprościć proces instalacji i uruchomienia CAN Simulatora na komputerze Wojtka (oraz innych użytkowników), przygotowano konfigurację Docker. Dzięki temu nie ma potrzeby ręcznego instalowania Pythona, bibliotek ani martenienia się o zależności systemowe.

11.2 Pliki konfiguracyjne

W głównym katalogu projektu znajdują się:

- `Dockerfile` – definicja obrazu.
- `docker-compose.yml` – uproszczone uruchamianie z mapowaniem wolumenów i zmiennych środowiskowych.
- `run_docker.sh` – skrypt pomocniczy budujący obraz i uruchamiający kontener.

11.3 Budowa obrazu

Obraz bazuje na `python:3.10-slim`. Instalowane są pakiety systemowe (`iproute2`, `can-utils`, `tk`) oraz zależności Pythona z `requirements.txt`. Aplikacja uruchamiana jest z uprawnieniami `-privileged`, aby móc tworzyć wirtualny interfejs `vcan0`.

```
1 FROM python:3.10-slim
2 RUN apt-get update && apt-get install -y --no-install-recommends \
3     iproute2 can-utils tk && rm -rf /var/lib/apt/lists/*
4 WORKDIR /app
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7 COPY . .
8 CMD ["python", "main.py"]
```

Listing 11.1: Dockerfile

11.4 Uruchomienie

Użytkownik wykonuje:

```
./run_docker.sh
```

lub

```
docker-compose up
```

Kontener uruchamia się w trybie **host** sieci, co pozwala mostkowi vCAN komunikować się z zewnętrznymi serwerami WebSocket. Wyświetlanie GUI realizowane jest poprzez współdzielenie gniazda X11 (`/tmp/.X11-unix`).

Rozdział 12

Mapa drogowa i wizja przyszłości

12.1 Podsumowanie dotychczasowych osiągnięć

Projekt CAN Simulator przeszedł kompleksową transformację – od prostego odtwarzacza logów do zaawansowanej platformy diagnostycznej. Zrealizowano 15 głównych etapów modernizacji, obejmujących m.in.:

- Reorganizację GUI.
- Pełną integrację DBC.
- Zdalny monitoring (WebSocket/WSS) i dwukierunkowy mostek vCAN.
- Powiadomienia Telegram i interfejs webowy.
- Wykrywanie anomalii w czasie rzeczywistym (autoenkoder).
- Nagrywanie sesji, testy jednostkowe i wsparcie Dockera.

12.2 Plany krótkoterminowe (Faza 2)

- **Obsługa reconnectu po stronie serwera** – automatyczne czyszczenie nieaktywnych sesji WebSocket.
- **Szyfrowanie plików konfiguracyjnych** – ochrona tokenów przechowywanych w `session.json`.
- **Watchdog dla mostka vCAN** – automatyczne odtwarzanie interfejsu `vcan0` po jego usunięciu.

12.3 Plany średnioterminowe (Faza 3)

- **Modelowanie profili sesji** – uczenie modeli "normalnego" zachowania i porównywanie z bieżącym ruchem.

- **Raporty okresowe (PDF)** – automatyczne generowanie i wysyłka e-mailem.
- **Harmonogram zadań** – możliwość ustawienia automatycznego nagrywania o określonych porach.

12.4 Plany doterminowe (Faza 4)

- **Wersja "headless"** – uruchomienie jako usługa systemowa (bez GUI) na Raspberry Pi.
- **Wsparcie dla CAN FD** – rozszerzenie parserów i interfejsu.
- **Instalator dla Windows** – plik .exe dla użytkowników niekorzystających z Linuksa.

12.5 Wizja przyszłości (Faza 5)

- **Uczenie federacyjne** – trenowanie modeli ML na danych z wielu instancji bez przesyłania surowych danych.
- **Cloud Dashboard** – centralny panel agregujący dane z floty pojazdów.
- **Integracja z bazami czasu rzeczywistego** (InfluxDB/TimescaleDB).
- **Symulacja odpowiedzi ECU** – emulacja modułów reagujących na zapytania diagnostyczne (UDS).

12.6 Zakończenie

CAN Simulator osiągnął poziom dojrzałości, który pozwala na wykorzystanie go w rzeczywistych zastosowaniach inżynierskich. Dalszy rozwój, zgodny z przedstawioną mapą drogową, uczyni z niego w pełni autonomiczną platformą analityczną, zdolną do predykcyjnego wykrywania usterek i wspomagania decyzji diagnostycznych.

Dodatek A

Kod Źródłowy mostka vCAN (bridge_client.py)

Poniżej przedstawiono pełny kod pliku `bridge_client.py` (wersja dwukierunkowa z obsługą auto-reconnect, filtrowania ID oraz wysyłania ramek z `vcan0` do serwera).

```
1 \#!/usr/bin/env python3
2 """
3 Mostek WebSocket -> wirtualny CAN (vcan0) z auto-reconnect i
4   filtrowaniem ID.
5 Dwukierunkowy: odbiera ramki z WebSocket -> vcan0 oraz wysyła
6   ramki z vcan0 -> WebSocket.
7 """
8
9 import json
10 import logging
11 import threading
12 import time
13 import subprocess
14 import sys
15 import argparse
16 from typing import List, Optional
17
18 import websocket
19
20 try:
21     import can
22     from can import ThreadSafeBus
23 except ImportError:
24     can = None
25     ThreadSafeBus = None
26
27 logging.basicConfig(level=logging.INFO, format='%(asctime)s [%(
28   levelname)s] %(message)s')
29 logger = logging.getLogger("CAN-Bridge")
30
31 class VCanBridge:
32     def __init__(self, url: str, token: Optional[str] = None,
33                 filter_ids: Optional[List[int]] = None,
34                 auto_reconnect: bool = True,
35                 reconnect_delay: int = 5,
```

```

34         status\_callback=None):
35     self.url = url
36     self.token = token
37     self.filter\_ids = set(filter\_ids) if filter\_ids else
        None
38     self.auto\_reconnect = auto\_reconnect
39     self.reconnect\_delay = reconnect\_delay
40     self.status\_callback = status\_callback
41
42     self.ws = None
43     self.bus = None
44     self.running = False
45     self.reconnect\_thread = None
46
47     self.sending\_to\_vcan = False    \# flaga zapobiegajÄc
        pÄŹtli zwrotnej
48     self.tx\_thread = None
49
50     self.\_setup\_vcan()
51
52     def \_setup\_vcan(self):
53         if can is None:
54             raise RuntimeError("python-can not installed; bridge
                cannot run.")
55         try:
56             subprocess.run(['ip', 'link', 'show', 'vcan0'], check=
                True, capture\_output=True)
57             logger.info("vcan0 juÅi istnieje.")
58         except subprocess.CalledProcessError:
59             logger.info("Tworzenie vcan0...")
60             subprocess.run(['sudo', 'ip', 'link', 'add', 'dev', '
                vcan0', 'type', 'vcan'], check=True)
61             subprocess.run(['sudo', 'ip', 'link', 'set', 'vcan0', '
                up'], check=True)
62             logger.info("vcan0 utworzony i wÄcÄczony.")
63             self.bus = ThreadSafeBus(channel='vcan0', interface='
                socketcan')
64
65     def \_should\_forward(self, can\_id: int) -> bool:
66         if self.filter\_ids is None:
67             return True
68         return can\_id in self.filter\_ids
69
70     def \_on\_message(self, ws, message):
71         try:
72             data = json.loads(message)
73             can\_id = int(data['id'], 16) if isinstance(data['id'],
                str) else data['id']
74
75             if not self.\_should\_forward(can\_id):
76                 logger.debug(f"PominiÄto ID 0x\{can\_id:X\} (
                    filtrowanie)")
77                 return
78
79             payload = bytes(data['data'])
80             is\_extended = data.get('is\_extended', False)
81             msg = can.Message(arbitration\_id=can\_id, data=payload
                , is\_extended\_id=is\_extended)

```

```

82         self.sending\_to\_vcan = True
83         self.bus.send(msg)
84         self.sending\_to\_vcan = False
85         if self.status\_callback:
86             self.status\_callback(f"Odebrano z serwera: \{msg\}
87                                     ")
88         except Exception as e:
89             logger.error(f"BŁĄD ramki: \{e\}")
90
91     def \_on\_error(self, ws, error):
92         logger.error(f"WebSocket error: \{error\}")
93         if self.status\_callback:
94             self.status\_callback(f"BŁĄD: \{error\}")
95
96     def \_on\_close(self, ws, code, msg):
97         logger.info("WebSocket zamknięty.")
98         if self.status\_callback:
99             self.status\_callback("Rozłączono")
100         if self.auto\_reconnect and self.running:
101             self.\_start\_reconnect()
102
103     def \_on\_open(self, ws):
104         logger.info("Połączono z serwerem.")
105         if self.token:
106             ws.send(self.token)
107         # Wyżliż listę dozwolonych ID (jeżeli ustawiono)
108         if self.filter\_ids is not None:
109             allowed\_msg = json.dumps(\{"allowed\_ids": list(self.
110                                     filter\_ids)\})
111             ws.send(allowed\_msg)
112             logger.info(f"Wysłano listę dozwolonych ID: \{self.
113                                     filter\_ids\}")
114
115         if self.status\_callback:
116             self.status\_callback("Połączono")
117
118     def \_start\_reconnect(self):
119         if self.reconnect\_thread and self.reconnect\_thread.is\
120             \_alive():
121             return
122         self.reconnect\_thread = threading.Thread(target=self.\
123             \_reconnect\_loop, daemon=True)
124         self.reconnect\_thread.start()
125
126     def \_reconnect\_loop(self):
127         while self.running:
128             logger.info(f"Próba ponownego połączenia za \{self.
129                                     reconnect\_delay\} s...")
130             time.sleep(self.reconnect\_delay)
131             if not self.running:
132                 break
133             try:
134                 self.ws = websocket.WebSocketApp(
135                     self.url,
136                     on\_open=self.\_on\_open,
137                     on\_message=self.\_on\_message,
138                     on\_error=self.\_on\_error,
139                     on\_close=self.\_on\_close

```

```

134         )
135         wst = threading.Thread(target=self.ws.run\_forever,
136                                daemon=True)
137         wst.start()
138         break
139     except Exception as e:
140         logger.error(f"BŁĄD reconnect: {e}")
141
142 def start(self):
143     self.running = True
144     self.ws = websocket.WebSocketApp(
145         self.url,
146         on\_open=self.\_on\_open,
147         on\_message=self.\_on\_message,
148         on\_error=self.\_on\_error,
149         on\_close=self.\_on\_close
150     )
151     wst = threading.Thread(target=self.ws.run\_forever, daemon=
152                             True)
153     wst.start()
154
155     \# Uruchom wÅłtek nadawczy (vcn -> WebSocket)
156     self.tx\_thread = threading.Thread(target=self.\_tx\_loop,
157                                         daemon=True)
158     self.tx\_thread.start()
159
160 def \_tx\_loop(self):
161     """WÅłtek odbierajÅłcy ramki z vcan0 i wysyÅłajÅłcy je
162     przez WebSocket."""
163     while self.running:
164         try:
165             msg = self.bus.recv(timeout=0.5)
166             if msg is None:
167                 continue
168             if self.sending\_to\_vcn:
169                 continue
170             if self.filter\_ids and msg.arbitration\_id not in
171                 self.filter\_ids:
172                 continue
173             if self.ws and self.ws.sock and self.ws.sock.
174                 connected:
175                 frame = \{
176                     "id": hex(msg.arbitration\_id),
177                     "data": list(msg.data),
178                     "is\_extended": msg.is\_extended\_id
179                 \}
180                 self.ws.send(json.dumps(frame))
181                 if self.status\_callback:
182                     self.status\_callback(f"WysÅłano do serwera
183                                           : {msg}")
184         except Exception as e:
185             logger.error(f"BŁĄD w pÅłtli TX: {e}")
186             time.sleep(0.1)
187
188 def stop(self):
189     self.running = False
190     if self.tx\_thread:
191         self.tx\_thread.join(timeout=1.0)

```

```

185         if self.ws:
186             self.ws.close()
187         if self.bus:
188             self.bus.shutdown()
189         if self.status\_callback:
190             self.status\_callback("Zatrzymano")
191
192
193 def main():
194     parser = argparse.ArgumentParser()
195     parser.add\_argument('--url', default='ws://localhost:8765')
196     parser.add\_argument('--token')
197     parser.add\_argument('--filter-ids', help='Lista ID CAN
198         oddzielonych przecinkami, np. 0x123,0x456')
199     parser.add\_argument('--no-auto-reconnect', action='store\_true',
200         help='Wyłącza automatyczne ponawianie')
201     args = parser.parse\_args()
202
203     filter\_ids = None
204     if args.filter\_ids:
205         try:
206             filter\_ids = [int(x.strip(), 16) if x.strip().
207                 startswith('0x') else int(x.strip())
208                 for x in args.filter\_ids.split(',')]
209         except ValueError:
210             logger.error("Nieprawidłowy format listy ID.")
211             sys.exit(1)
212
213     bridge = VCanBridge(
214         url=args.url,
215         token=args.token,
216         filter\_ids=filter\_ids,
217         auto\_reconnect=not args.no\_auto\_reconnect,
218         status\_callback=print
219     )
220     bridge.start()
221     try:
222         while True:
223             time.sleep(1)
224     except KeyboardInterrupt:
225         bridge.stop()
226
227 if \_\_name\_\_ == "\_\_main\_\_":
228     main()

```


Dodatek B

Kod Źródłowy serwera WebSocket (server.py)

Poniżej przedstawiono pełny kod pliku `broadcaster/server.py`. Serwer obsługuje tokeny, SSL, autoryzację ID, logowanie do pliku, powiadomienia Telegram oraz opcjonalny interfejs webowy (dashboard).

```
1  """
2  Serwer WebSocket do przesyłania ramek CAN w czasie rzeczywistym.
3  Obsługuje opcjonalny token autoryzacyjny, autoryzację ID,
4  logowanie do pliku,
5  powiadomienia Telegram oraz interfejs webowy (dashboard).
6  """
7
8  import asyncio
9  import json
10 import logging
11 from datetime import datetime
12 from typing import Set, Optional
13 import ssl
14 import urllib.request
15 import urllib.parse
16 import os
17
18 import websockets
19 from websockets.server import WebSocketServerProtocol
20
21 \# aiohttp jest opcjonalne - jeśli nie ma, dashboard HTTP nie
22   będzie dostępny
23
24 try:
25     from aiohttp import web
26     AIOHTTP_AVAILABLE = True
27 except ImportError:
28     AIOHTTP_AVAILABLE = False
29     web = None
30
31 logger = logging.getLogger(__name__)
32
33 class CANWebSocketServer:
34     def __init__(self, host: str = "0.0.0.0", port: int = 8765,
35                  token: Optional[str] = None, ssl_context:
36                      Optional[ssl.SSLContext] = None):
```

```

34     self.host = host
35     self.port = port
36     self.token = token
37     self.ssl\_context = ssl\_context
38     self.\_server = None
39     self.\_clients: Set[WebSocketServerProtocol] = set()
40
41     self.can\_interface = None
42     self.allowed\_client\_ids = None
43     self.incoming\_filter\_ids = None
44     self.log\_to\_file = False
45     self.log\_file\_path = "remote\_operations.log"
46
47     self.telegram\_token = None
48     self.telegram\_chat\_id = None
49
50     self.enable\_http = False
51     self.http\_port = 8080
52     self.http\_app = None
53     self.http\_runner = None
54
55     self.\_running = False
56
57     def \_send\_telegram(self, message):
58         if not self.telegram\_token or not self.telegram\_chat\_id:
59             return
60         try:
61             url = f"https://api.telegram.org/bot\{self.telegram\_
62                 \_token\}/sendMessage"
63             data = urllib.parse.urlencode(\{
64                 "chat\_id": self.telegram\_chat\_id,
65                 "text": f"0§ŽŮ CAN Simulator\textbackslash n\{
66                     message\}",
67                 "parse\_mode": "HTML"
68             \}).encode("utf-8")
69             req = urllib.request.Request(url, data=data)
70             with urllib.request.urlopen(req, timeout=5) as resp:
71                 if resp.getcode() != 200:
72                     logger.warning(f"Telegram odpowiedziaĀ kodem
73                         \{resp.getcode()\}")
74         except Exception as e:
75             logger.error(f"BĀĀĒd wysyĀcia do Telegram: \{e\}")
76
77     async def \_handler(self, websocket: WebSocketServerProtocol):
78         logger.info(f"Nowy klient: \{websocket.remote\_address\}")
79
80         if self.token:
81             try:
82                 msg = await asyncio.wait\_for(websocket.recv(),
83                     timeout=5.0)
84                 if msg != self.token:
85                     logger.warning(f"NieprawidĀowy token od \{
86                         websocket.remote\_address\}")
87                     await websocket.close(1008, "Invalid token")
88                     return
89                 logger.info(f"Token zaakceptowany od \{websocket.
90                     remote\_address\}")
91             except asyncio.TimeoutError:

```

```

86         logger.warning(f"Timeout tokena od \{websocket.
87             remote\_address\}")
88         await websocket.close(1008, "Token timeout")
89         return
90     if self.allowed\_client\_ids is not None:
91         try:
92             msg = await asyncio.wait\_for(websocket.recv(),
93                 timeout=5.0)
94             data = json.loads(msg)
95             client\_ids = set(data.get("allowed\_ids", []))
96             if not client\_ids.issubset(self.allowed\_client\_
97                 _ids):
98                 logger.warning(f"Klient \{websocket.remote\
99                     _address\} próbował uÅyć niedozwolonych
100                     ID")
101                 await websocket.close(1008, "Forbidden IDs")
102                 return
103                 websocket.client\_allowed\_ids = client\_ids
104                 logger.info(f"Klient \{websocket.remote\_address\}
105                     autoryzowany z ID: \{client\_ids\}")
106         except (asyncio.TimeoutError, json.JSONDecodeError):
107             logger.warning(f"Brak autoryzacji ID od \{
108                 websocket.remote\_address\}")
109             await websocket.close(1008, "Invalid ID list")
110             return
111
112     self.\_clients.add(websocket)
113     logger.info(f"Klient dodany, Åcznie: \{len(self.\
114         _clients)\}")
115     self.\_send\_telegram(f"§ Klient poÅczony: \{
116         websocket.remote\_address\}")
117
118     try:
119         async for message in websocket:
120             await self.\_handle\_incoming\_frame(websocket,
121                 message)
122     except websockets.exceptions.ConnectionClosed:
123         logger.info(f"PoÅczenie zamniÅte przez klienta \{
124             websocket.remote\_address\}")
125     finally:
126         self.\_clients.remove(websocket)
127         logger.info(f"Klient rozÅczony, pozostaÅo: \{len(
128             self.\_clients)\}")
129         self.\_send\_telegram(f"§ Klient rozÅczony: \{
130             websocket.remote\_address\}")
131
132     async def \_handle\_incoming\_frame(self, websocket, message):
133         if not self.incoming\_filter\_ids:
134             logger.debug("Brak filtru przychodzącego Å ramka
135                 odrzucona.")
136             return
137         try:
138             data = json.loads(message)
139             can\_id = int(data['id'], 16) if isinstance(data['id'],
140                 str) else data['id']
141             if can\_id not in self.incoming\_filter\_ids:
142                 logger.debug(f"ID 0x\{can\_id:X\} nie przechodzi

```

```

129         filtra przychodzÄcego.")
130     return
131     payload = bytes(data['data'])
132     is\_extended = data.get('is\_extended', False)
133     if self.can\_interface and self.can\_interface.
        connected:
134         success, msg = self.can\_interface.send\_frame(can\_
            _id, payload, is\_extended)
135         if success:
136             logger.info(f"Wysłano na CAN: ID=0x\{can\_id:X
                \}, data=\{payload.hex().upper()\}")
137             if self.log\_to\_file:
138                 self.\_log\_to\_file(f"RX from \{websocket.
                    remote\_address\}: \{message\}")
139             else:
140                 logger.error(f"Błąd wysyłania na CAN: \{msg
                    \}")
141                 self.\_send\_telegram(f"âiÑ Błąd wysyłania
                    na CAN: \{msg\}")
142         else:
143             logger.warning("CAN niepodłączony âŞ ramka
                odrzucona.")
144     except Exception as e:
145         logger.error(f"Błąd przetwarzania ramki od klienta:
            \{e\}")
146
147 def \_create\_http\_app(self):
148     if not AIOHTTP\_AVAILABLE:
149         return None
150     app = web.Application()
151     web\_dir = os.path.join(os.path.dirname(os.path.dirname(\_
        _file\_)), 'web\_dashboard')
152     if os.path.isdir(web\_dir):
153         app.router.add\_static('/', web\_dir, show\_index=True)
154     return app
155
156 async def start(self):
157     if self.\_running:
158         return
159     logger.info(f"Uruchamianie serwera WebSocket na \{self.host
        \}:\{self.port\} (SSL: \{self.ssl\_context is not None
        \})")
160     self.\_server = await websockets.serve(
161         self.\_handler, self.host, self.port, ssl=self.ssl\
        _context
162     )
163     self.\_running = True
164
165 if self.enable\_http:
166     self.http\_app = self.\_create\_http\_app()
167     if self.http\_app:
168         runner = web.AppRunner(self.http\_app)
169         await runner.setup()
170         site = web.TCPSite(runner, self.host, self.http\
        _port)
171         await site.start()
172         self.http\_runner = runner
173         logger.info(f"Interfejs webowy dostÄpny na http

```

```

        ://\{self.host\}:\{self.http\_port\}")

173
174 async def stop(self):
175     if not self.\_running:
176         return
177     logger.info("Zatrzymywanie serwera...")
178     self.\_server.close()
179     await self.\_server.wait\_closed()
180     for client in list(self.\_clients):
181         await client.close()
182     self.\_clients.clear()
183
184     if self.http\_runner:
185         await self.http\_runner.cleanup()
186         self.http\_runner = None
187
188     self.\_running = False
189     logger.info("Serwer zatrzymany")
190
191 def \_log\_to\_file(self, msg):
192     try:
193         with open(self.log\_file\_path, 'a', encoding='utf-8')
194             as f:
195                 f.write(f"\{datetime.now().isoformat()\} \{msg\}\
196                     textbackslash n")
197     except Exception as e:
198         logger.error(f"BŁĄD zapisu do pliku logu: \{e\}")
199
200 def broadcast\_frame(self, frame: dict):
201     if not self.\_running or not self.\_clients:
202         return
203     message = json.dumps(frame)
204     for client in list(self.\_clients):
205         try:
206             asyncio.create\_task(client.send(message))
207         except Exception as e:
208             logger.error(f"BŁĄD wysyłania: \{e\}")
209
210 async def broadcast\_frame\_async(self, frame: dict):
211     if not self.\_running or not self.\_clients:
212         return
213     message = json.dumps(frame)
214
215     if self.log\_to\_file:
216         self.\_log\_to\_file(f"TX to clients: \{message\}")
217
218     for client in list(self.\_clients):
219         try:
220             await client.send(message)
221         except Exception as e:
222             logger.error(f"BŁĄD wysyłania async: \{e\}")
223
224 @property
225 def client\_count(self) -> int:
226     return len(self.\_clients)
227
228 @property
229 def is\_running(self) -> bool:

```

228 `return self._running`